

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du cours. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Commencer par remplir nom, prénom et numéro étudiant.

Sauf indication contraire, les notations attendues dans cette partie sont celles du CM.

Certains n'ont pas lu le sujet en entier.

1. Répondre aux questions suivantes en justifiant brièvement les réponses.

- Soit $\mathcal{E}$ un ensemble fini. Que signifient $\text{card}(\mathcal{E})$ ou $||\mathcal{E}||$?

Le cardinal de $\mathcal{E}$, soit son nombre d'éléments.

Ce concept est facile à définir si $\mathcal{E}$ est fini, plus difficile sinon (voir picopoly). Mais pour la suite de cette question, tout est fini.

- Soit $\mathcal{M} = (\Sigma, Q, q_0, F, \delta)$ un automate. Quel est le maximum de $\text{card}(F)$?

Par définition, F est un sous-ensemble de Q ,
donc $\text{card}(F) \leq \text{card}(Q)$.

- Soit $\mathcal{M} = (\Sigma, Q, q_0, F, \delta)$ un automate possiblement non-déterministe. Quel est le maximum de $\text{card}(\delta)$?

Par définition, δ est un sous-ensemble du produit cartésien $Q \times \Sigma \times Q$,
donc $\text{card}(\delta) \leq \text{card}(Q)^2 \times \text{card}(\Sigma)$.

- Soit $\mathcal{M} = (\Sigma, Q, q_0, F, \delta)$ un automate déterministe. Quel est le maximum de $\text{card}(\delta)$?

Si $\mathcal{M}$ est déterministe alors δ est un sous-ensemble de $(Q \times \Sigma) \rightarrow Q$. En d'autres termes, il y a un seul $q_{\text{arrivée}}$ (le second Q) pour chaque $(q_{\text{départ}}, \sigma)$.

Donc, $\text{card}(\delta) \leq \text{card}(Q) \times \text{card}(\Sigma)$.

Et c'est pourquoi on peut en faire une table à 2 entrées dans l'algorithme d'acceptation donné en cours.

On souhaite formaliser les langages $\mathcal{L}_{LD}$ des listes de chiffres et le langage $\mathcal{L}_{LLD}$ des listes de listes de chiffres. Les listes seront représentées dans le style Python, avec des crochets pour marquer le début et la fin, et des virgules pour séparer les éléments : par exemple, $[[1, 2, 3], [4, 5]]$. On ignorera les espaces.

2. Rayer dans ce qui suit les contre-exemples de $\mathcal{L}_{LLD}$.

- $[]$

C'est bien une liste de listes d'entiers, mais vide.

Certains n'aiment pas les listes vides. Qu'ils le disent, et soient cohérents tout au long de cette séquence de questions. Et qu'ils se rappellent quand même de $[]$ en Python, qui est bien une liste vide !

- $[[]]$

Une liste non vide contenant une liste vide d'entiers.

Noter que la différence avec le précédent cas est de même nature qu'entre $\emptyset$ et $\{\varepsilon\}$.

- $[[] , []]$

Une liste non vide contenant des listes vides d'entiers.

- ~~$[[1], 2]$~~

Une liste dont un élément est une liste d'entiers, mais l'autre un entier, donc pas une liste de listes d'entiers.

- $[[1], [2, 3]]$

Une liste de listes d'entiers.

- ~~$[1, 2]$~~

Une liste d'entiers. Donc pas une liste de listes d'entiers.

- ~~$[[[1, 2]]]$~~

Une liste de listes de listes d'entiers ! Donc pas une liste de listes d'entiers.

- ~~$[[11], [22, 33]]$~~

Une liste de listes de nombres ! Donc pas une liste de listes d'entiers.

Ne jamais confondre chiffres et nombres. Les chiffres sont des symboles, ex. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, et les (représentations de) nombres sont des mots finis formés avec ces symboles.

3. Proposer une formalisation de $\mathcal{L}_{LLD}$ par une expression régulière.

$$\begin{aligned} RE_D &= (0|1|2|3|4|5|6|7|8|9) \\ RE_{LD} &= ('[' \bullet RE_D \bullet (';' \bullet RE_D)^* \bullet ']') | ('[' \bullet ']') \\ RE_{LLD} &= ('[' \bullet RE_{LD} \bullet (';' \bullet RE_{LD})^* \bullet ']') | ('[' \bullet ']') \end{aligned}$$

Deux fois des trucs séparés par des machins (en **vert**) ! Des trucs différents, mais les mêmes machins.

Le cas des listes vides peut être traité à l'extérieur du cas général, comme ici en **rouge**, et ça laisse le cas général intact, ou bien en modifiant le cas général. On conservera ce code de couleur dans les réponses qui suivent qui traitent du même sujet : **vert** le cas général, **rouge** le cas vide.

Ne pas se dire qu'il est impossible de formaliser $\mathcal{L}_{LLD}$ par une expression régulière à cause de l'imbrication des crochets. L'impossibilité ne vaut que lorsque les imbrications sont non bornées, mais ici il n'y a que 2 niveaux d'imbrication. Se rappeler que

$$\{a^n b^n \mid 0 \leq n\}$$

n'est pas rationnel (voir Lemme de l'étoile), mais que

$$\{a^n b^n \mid 0 \leq n \leq L\}$$

l'est pour tout L. Je ne poserais pas une question comme « Proposer une formalisation ... » si je savais qu'il n'y en a pas. Je ne suis pas malveillant !

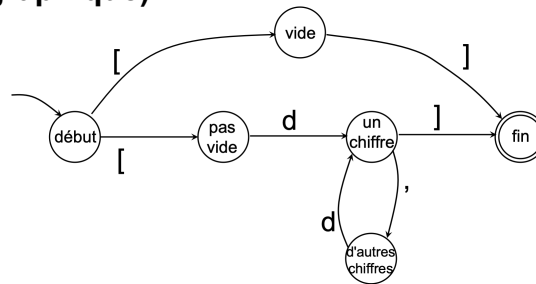
Beaucoup trop de réponses fausses parce que vous essayez de donner une réponse en un coup. C'est presque impossible à faire quand on voit que RE_{LD} est déjà assez complexe et doit être répétée deux fois dans RE_{LLD} . Combien de fois vous ai-je dit de construire les réponses par morceau en donnant des noms aux morceaux ? Et je vous ai même dit que cela pouvait être la présentation finale de votre réponse. Il n'est pas nécessaire de produire l'expression consolidée.

Beaucoup trop de réponses qui n'étaient pas cohérentes avec ce que vous aviez désigné comme exemple et contre-exemple à la question 2. La question 2 n'était pas là pour vous embêter, mais pour vous obliger à vous poser une minute pour réfléchir à différents cas.

Encore trop de confusions de notation. Ici, nous sommes dans un **CC de CM**, et il commence par la phrase « **Sauf indication contraire, les notations attendues dans cette partie sont celles du CM** », et dans le retour du CC d'avant j'ai consacré une ½ page à la question du respect des notations. Que peut-on dire de plus ?

Certains ont bien vu qu'il fallait traiter les cas vides et que $\text{truc} \bullet (\text{machin} \bullet \text{truc})^*$ ne le faisait pas. Alors ils ont eu une crise d'étoile ! Et on répondit $(\text{truc} \bullet (\text{machin} \bullet \text{truc})^*)^*$, mais ça ne marche pas.

4. Proposer une formalisation de $\mathcal{L}_{LD}$ (pas $\mathcal{L}_{LLD}$!) par un automate fini (ne donner que sa présentation graphique).



Cette solution est non-déterministe (état « début »). Mais je vous ai expliqué de nombreuses fois que les solutions non-déterministes sont souvent plus simples à formuler que les solutions déterministes, et que si les consignes ne demandaient pas de réponses déterministes il ne fallait pas se priver. Cet automate n'est pas non plus minimal, mais il me semble raconter une histoire plus simple que celui où la flèche ',' retourne vers l'état « pas vide » (qui est bien sûr tout à fait correct).

Ici, ce qui traite le cas vide est mis à l'extérieur du cas général, comme pour l'expression régulière. De cette façon, le cas général n'est pas pollué par le cas particulier de la liste vide, et il reste facile à lire.

5. Proposer une formalisation de $\mathcal{L}_{LLD}$ par une grammaire algébrique.

des listes, vides ou pas, de listes de chiffres

$LLD \rightarrow '[' LD\text{virg} ']'$ # cas non vide

$LLD \rightarrow \text{'[' ']'}$ # cas vide

$LD\text{virg} \rightarrow LD$ # des LD séparés par des ','

$LD\text{virg} \rightarrow LD\text{virg} ',' LD$

même chose pour des listes, vides ou pas, de chiffres

$LD \rightarrow '[' D\text{virg} ']'$

$LD \rightarrow \text{'[' ']'}$

$D\text{virg} \rightarrow D$

$D\text{virg} \rightarrow D\text{virg} ',' D$

$D \rightarrow$ les chiffres de 0 à 9

Comme pour 3, on a 2 fois des trucs séparés par des machins, mais dans le style grammaire algébrique cette fois (en **vert**). On en a vu des tas dans les CM et TD.

Il y a des critères de forme qui distinguent à coup sûr les solutions qui ne peuvent pas marcher. Par exemple, on doit voir 2 endroits qui engendrent des '[' et ']' et des ','. Mais il y a aussi des critères de formes qui distinguent presque certainement des solutions bancales. Par exemple, les '[' et ']' de même niveau devraient figurer dans le même

corps de règle. Si vous ne respecter pas ça c'est l'échec presque assuré. C'est vraiment en cela que les grammaires algébriques se distinguent des automates finis. Elles permettent de dire facilement « À chaque fois que j'ouvre un '[', je le ferme un peu plus loin » dans le mot généré. C'est pour ça qu'à la question 4 je n'ai demandé que l'automate pour $\mathcal{L}_{LD}$, et pas pour $\mathcal{L}_{LLD}$. L'automate pour $\mathcal{L}_{LLD}$ est vraiment horrible à concevoir.

6. Aurait-on pu proposer une formalisation de $\mathcal{L}_{LLD}$ par une grammaire régulière ?

Oui, car les grammaires régulières engendrent tous les langages rationnels (et rien qu'eux) ; et $\mathcal{L}_{LLD}$ est rationnel (voir la question 3) puisque vous avez pu le décrire par une expression régulière.

Rappel : une question oui/non appelle toujours une réponse « Oui, parce que ... » ou « Non, parce que ... ». Sinon, autant jouer le CC aux dés. Ici, j'attends un oui ferme, et il n'y a pas vraiment de façon raisonnable de répondre non. Mais il est des cas, où une réponse fausse peut avoir une justification intéressante qui vous sauverait quelques points.

7. Rappeler la définition de l'ambiguïté pour les grammaires algébriques.

Une grammaire est ambiguë si elle peut produire des arbres de dérivation différents, mais qui ont le même mot des feuilles.

Ici, il faut être extrêmement précis. Ce qui est source d'ambiguïté c'est de produire des arbres de dérivation différents pour le même mot engendré. Avoir des dérivations différentes du même mot engendré n'est absolument pas un problème ; c'est même souvent la règle. Revoir le TD là-dessus.

Ici, « ambiguïté » est une notion formelle qui a une définition formelle. On lui a donné ce nom-là par analogie avec l'ambiguïté informelle, mais il ne faut surtout pas se contenter du sens de l'ambiguïté informelle. Les sciences sont pleines de notions formelles à qui on a donné un nom venant du sens courant. Et la Théorie des Langages ne fait pas exception : lettre, mot, automate, grammaire, ambiguïté, arbre, etc.

8. Pourquoi l'ambiguïté doit-elle être évitée ? Donner un exemple.

L'ambiguïté doit être évitée car la sémantique, c'est-à-dire le sens, dépend souvent de la structure syntaxique que l'on donne aux mots engendrés. L'exemple vu en cours était 3-2-1 selon qu'on le lisait comme (3-2) -1 ou 3-(2-1). La petite école nous a tous appris à n'accepter que la première lecture, mais la grammaire naïve vue en cours engendrait les deux.

9. Qu'est-ce qui distingue les grammaires algébriques étendues ?

Les grammaires algébriques étendues permettent d'utiliser tous les constructeurs des expressions régulières dans le corps des règles de grammaire.

Beaucoup d'entre vous ont répondu en disant juste que les grammaires algébriques étendues permettent d'exprimer plus facilement des phénomènes récurrents comme les « trucs séparés par des machines ». Mais ça n'est pas ce qui distingue les grammaires algébriques étendues. C'est juste une bonne propriété de cette extension.

10. Proposer une formalisation de $\mathcal{L}_{LLD}$ par une grammaire algébrique étendue.

$LLD \rightarrow '[\textcolor{teal}{LD} (; \textcolor{teal}{LD})^*]' \mid \textcolor{red}{'T' 'T'}$

$LD \rightarrow '[\textcolor{teal}{D} (; \textcolor{teal}{D})^*]' \mid \textcolor{red}{'T' 'T'}$

$D \rightarrow$ les chiffres de 0 à 9

Ça ressemble beaucoup à la réponse à la question 3 ! En fait, le vocabulaire non-terminal des grammaires ressemble beaucoup aux noms qu'on donne aux sous-expressions algébriques.

Et toujours des trucs séparés par des machines (en **vert**) !

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du TP. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Commencer par remplir le cadre nom, prénom et numéro étudiant.

Sauf indication contraire, les notations attendues dans cette partie sont celles des TP.

Dans cette notation, les lettres terminales sont écrites sans quotes ('), la concaténation sans point (•), et les crochets ([et]) sont des caractères spéciaux.

1. Décrire la notation $\{n, m\}$ du package `regex`.
2. Que signifie $[25-79]$? Pourquoi ?

Les questions 1 et 2 ont déjà été posées lors du premier CC. Et des éléments de réponse ont été publiés. C'est donc une prime à ceux qui lisent attentivement les comptes-rendus de contrôle. Et une déprime pour les autres.

3. Proposer une expression régulière de `regex` qui décrit la notation des nombres entiers en base 10. La seule notation qui commence par 0 devra être 0.

```
re_base10 = "0|([1-9][0-9]*)"
```

4. Proposer une version `regex` de la réponse à la question 3 de la partie CM.

```
re_LD = "(\[\d(,\d)*\])|(\[\])"
```

```
RE_LD = f"(\[{re_LD}(,{re_LD})*\])|(\[\])"
```

Toujours des trucs séparés par des machines (en **vert**).

Faire attention aux notations concrètes du package `regex` (celui utilisé en TP) . En particulier, les `[` et `]` qui sont des caractères spéciaux. Pour les utiliser pour eux-mêmes, il faut les préfixer par le caractère d'échappement `\`.

Peut-être à cause de la question 1, certains n'ont pas osé écrire des étoiles, et ont mis des $\{0, \}$ à la place. Pourquoi pas, mais surtout ne pas écrire $\{1, \}$ pour l'étoile. Et tout ça ce sont des exemples de $\{n, \}$ que beaucoup d'ont pas commenté dans leur réponse à la question 1. Et attention, $?$ ne remplace pas du tout l'étoile.

Une autre façon de noter le cas vide :

```
re_LD = "(\\[(\\d(,\\d)*)?\\])"
RE_LLD = f"(\\[(\\{re_LD\\}(,\\{re_LD\\})*)?\\])"
```

L'un d'entre vous a écrit une étoile dans le style araignée écrasée, mais il a quand même eu la gentillesse ou la présence d'esprit d'écrire « étoile » à côté. Merci !

5. Proposer une version automata-lib de la réponse à la question 4 de la partie CM.

```
auto_ld = NFA(
    states={'début', 'non vide', 'vide',
            'un chiffre', 'autres chiffres'},
    input_symbols={'[', 'd', ','},
    initial_state='début',
    final_states={'fin'},
    transitions={
        'début': {'[': {'vide', 'pas vide'}},
        'vide': {']': {'fin'}},
        'début': {'d': {'un chiffre'}},
        'premier chiffre': {',': {'autres chiffres'},
                           ']' : {'fin'}},
        'autres chiffres': {'d': {'premier chiffre'}}
    })
```

Ce qui est important ici est d'avoir utilisé le constructeur NFA plutôt que DFA, puisque l'automate est non-déterministe. Et cela se traduit concrètement par le fait que la troisième composante de chaque transition est un ensemble d'état, et pas un état. Évidemment, ceux qui auront proposé un automate déterministe auront pu utiliser le constructeur DFA pour une notation plus compacte, mais le constructeur NFA pouvait encore être utilisé.

Comme beaucoup de vos automates de la question 4 de la partie CM étaient faux, je n'ai pris en compte que le fait que vos automates version automata-lib soient syntaxiquement corrects.

Plusieurs d'entre vous ont répondu d'une façon franchement bizarre à cette question. La question 4 de la partie CM de ce contrôle doit bien consister en un automate fini, déterministe ou non selon votre préférence. Alors pourquoi proposer d'utiliser une méthode toAutomaton d'une librairie pas utilisée en TP, pour un langage pas utilisé en TP, et qui traduit une expression régulière en un automate ? C'est vraiment bien d'aller voir plus loin que le cours et ses TD et TP, mais ça doit rester pour votre culture personnelle. Et la question demandait bien de partir de la réponse à la question 4 de la partie CM, pas de partir de la réponse à la question 3.

6. Rappeler ce que sont DFA et NFA pour le package automata-lib.

DFA et NFA sont respectivement les constructeurs des automates déterministes et possiblement non déterministes.

C'est une nuance importante car la représentation des transitions n'est pas la même dans les deux cas :

- transitions de DFA de type `dict(Q, dict( $\Sigma$ , Q))` ;
- et transitions de NFA de type `dict(Q, dict( $\Sigma$ , ens(Q)))` .

7. Décrire l'utilisation de la méthode `from-nfa` du package automata-lib.

La méthode `from-nfa` s'applique à un automate NFA et retourne un automate DFA qui reconnaît le même langage. Elle accepte aussi un paramètre qui détermine si l'automate résultat est minimisé :

```
dfa = DFA.from-nfa(nfa, minimize)
```

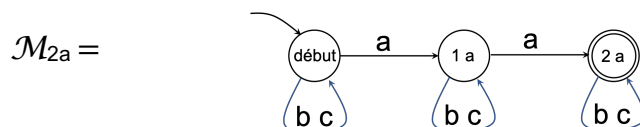
Beaucoup d'entre vous ont juste décrit ce que faisait la méthode, et pas du tout son utilisation : à quoi l'applique-t-on, avec quels paramètres. Même chose pour la question 8.

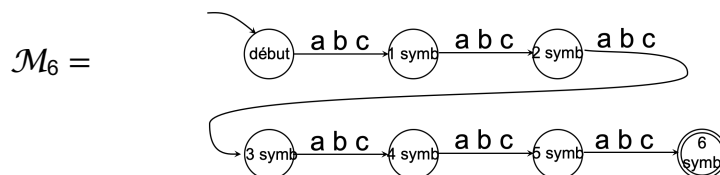
8. Décrire l'utilisation de la méthode `intersection` du package automata-lib.

On applique la méthode `intersection` à un automate, $\mathcal{M}_1$, avec pour premier paramètre un second automate, $\mathcal{M}_2$: $\mathcal{M}_1.intersection(\mathcal{M}_2)$. La méthode calcule un automate $\mathcal{M}$ tel que $\mathcal{L}(\mathcal{M}) = \mathcal{L}(\mathcal{M}_1) \cap \mathcal{L}(\mathcal{M}_2)$. La méthode `intersection` accepte un second paramètre, booléen, qui sert à spécifier si l'automate résultat doit être minimisé ou non.

Soit $\Sigma = \{a, b, c\}$. Soit $\mathcal{M}_{2a}$, $\mathcal{M}_{2b}$, $\mathcal{M}_{2c}$ les automates qui reconnaissent respectivement les langages des mots qui contiennent 2 a, 2 b ou 2 c, et $\mathcal{M}_6$ l'automate qui reconnaît le langage des mots de 6 symboles.

9. Donner la représentation graphique des automates $\mathcal{M}_{2a}$ et $\mathcal{M}_6$.





Attention ! La consigne « contenir 2 a » ne signifie pas que ces 2 a soient l'un à côté de l'autre, ou calés au début ou à la fin, ou même que le mot soit constitué de exactement 2 a. Vous avez tendance à sur-interpréter les consignes. « Contenir 2 a » c'est « contenir 2 a », rien de plus, rien de moins. On en a parlé en long et en large en TD. On peut bien sûr discuter de si « contenir 2 a » signifie « contenir au moins 2 a » ou « contenir exactement 2 a ». Mon automate ci-dessus implémente la vision stricte, « contenir exactement 2 a », comme quand vous dites « J'ai fait 2 fautes à mon examen » ou « J'ai mis 2 œufs dans mon gâteau » ou « Ma moto consomme 2 litres au cent » ou « Il y a 2 ampoules à remplacer » ou « Il reste 2 places ». Juste pour voir, cherchez une situation de la vie courante où la communication est sincère et où l'interprétation large, « au moins 2 », serait la bonne.

Sinon, beaucoup trop d'automates trop désinvoltes : pas d'état final, pas de symbole pour étiqueter les transitions. Ces choses sont tellement structurantes pour définir un automate qu'on n'a pas le droit de les négliger. Pensez juste qu'un automate sans état final est tout à fait légal, mais que ça signifie un automate qui n'accepte aucun mot, et donc un automate qui reconnaît le langage vide, et donc un automate très faux pour la question qui nous concerne.

10. Construire avec l'aide du package automata-lib un automate qui reconnaît le langage des mots de 6 lettres qui contiennent 2 a, 2 b et 2 c.

$\mathcal{M}_{2a2b} = \mathcal{M}_{2a}.\text{intersection}(\mathcal{M}_{2b})$

$\mathcal{M}_{2a2b2c} = \mathcal{M}_{2a2b}.\text{intersection}(\mathcal{M}_{2c})$

$\mathcal{M} = \mathcal{M}_{2a2b2c}.\text{intersection}(\mathcal{M}_6, \text{true})$ # pour minimiser $\mathcal{M}$

Ici, la case était bien trop petite pour y faire rentrer un automate qui fait plusieurs dizaines d'états. Heureusement que le package offre la méthode `intersection` pour construire un tel automate sans se tromper.

Bilan

Toujours trop de réponses réflexes à une question sans avoir lu les autres questions. Quand vous vous en rendez compte et le signalez par des flèches, je ne décompte pas de points, mais quand c'est votre dernier mot, je vois juste des réponses absurdes. On en a déjà parlé. Ce n'est pas une question de Théorie des Langages, c'est une question de « Profession : étudiant ». Dans le bon exercice de votre métier d'étudiant vous devez lire les sujets d'examen en entier avant de commencer à y répondre. Un peu perturbant : un étudiant qui ne s'était pas trompé de case a rajouté des flèches pour dire qu'il s'était trompé !

Trop d'étudiants qui n'ont su écrire ni une expression régulière, ni un automate, ni une grammaire ; qu'ont-ils étudié cet hiver ? Trop d'étudiants qui n'ont su écrire ni une regex ni un NSA ; qu'ont-ils fait en TP cet hiver ?